

AI Study Assistant: Design and Implementation of an NLP-Based Note Retrieval and Quiz Generation System

Project submitted to the
SRM University – AP, Andhra Pradesh
for the partial fulfillment of the requirements to award the degree of

Bachelor of Technology

In

Computer Science and Engineering

School of Engineering and Sciences

Submitted by

K.Harsha - AP23110011003

SB.Navaneeth - AP23110011106



SRM University–AP
Neerukonda, Mangalagiri, Guntur
Andhra Pradesh – 522 240
[December, 2025]

AI Study Assistant

Project Description:

AI Study Assistant is an innovative Generative AI-based learning system designed to help students efficiently manage their study materials, revise concepts, and improve their understanding through interactive tools. Many students face challenges in organizing notes, revising large amounts of content, and testing their knowledge due to scattered resources and lack of intelligent support. This system addresses these issues by enabling students to upload notes and assignments, retrieve relevant information, and generate quizzes automatically for effective learning.

Powered by modern NLP techniques and embedding-based retrieval, the system analyzes uploaded study materials and user queries to provide meaningful and context-aware responses. It uses intelligent search mechanisms to fetch relevant content and supports quiz generation to enhance revision and self-assessment. By integrating tool-based logic, the assistant dynamically decides whether to retrieve notes or generate quizzes based on user input, making the learning process more interactive and efficient.

Built using Python, FastAPI, FAISS, and Sentence Transformers, the platform combines multiple intelligent components into a unified system. The note retrieval module stores and searches content using vector embeddings, allowing accurate and fast access to relevant information. The quiz generation module creates structured questions and answers from the uploaded content, helping students test their understanding. The system interface provides a simple and user-friendly way to upload files, ask questions, and generate quizzes seamlessly.

Additionally, the system follows a tool orchestration approach where different functionalities such as search and quiz generation are handled through dedicated tools. This makes the system modular, scalable, and closer to real-world LLM-based architectures. By combining retrieval systems with intelligent processing, the AI Study Assistant transforms traditional learning methods into a smarter, more interactive, and adaptive experience.

Overall, the system helps students save time, improve revision efficiency, and gain better insights into their learning process. It acts as a smart study companion that supports continuous learning and makes education more accessible and effective.

Scenario 1: Intelligent Note Retrieval and Study Planning

The AI Study Assistant analyzes the query and dynamically selects the appropriate tool, such as `search_notes()`, to provide accurate and context-based results. It returns important information that helps students focus on key concepts, making their study process more organized and efficient. This enables students to quickly revise topics without manually searching through large volumes of notes..

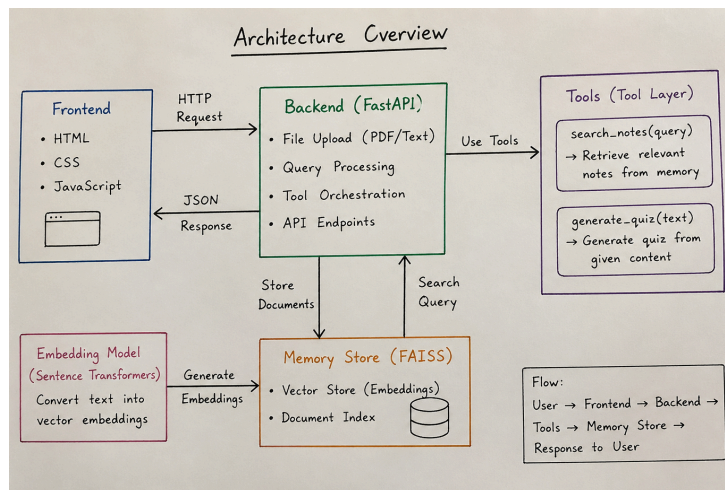
Scenario 2: Quiz Generation and Interactive Learning

Through the system interface, users request quizzes based on their uploaded notes or specific topics. The assistant processes the retrieved content and uses the `generate_quiz()` tool to create structured questions and answers for self-assessment. Based on the user's input and available data, the system generates relevant quiz content that helps in testing understanding and improving retention. This integration ensures students receive quick and context-based practice material—supporting effective revision and continuous learning through an AI-assisted approach.

Scenario 3: Adaptive Response Handling and Learning Feedback

As users interact with the system by asking questions or generating quizzes, the AI Study Assistant continuously processes queries and retrieves relevant information from stored notes. If limited or no data is found, the system provides fallback responses to ensure smooth interaction. Based on user queries and available content, it dynamically selects appropriate tools to deliver accurate results. This approach helps maintain consistency in learning and ensures students receive meaningful feedback, supporting better understanding and continuous improvement.

Architecture Overview :



LifeOptimizer is a modular, AI-driven personal productivity platform designed to generate personalized routines, track habits, and curate motivational content. It leverages a Flask-based backend integrated with Google's Gemini 1.5 Flash model to analyze user data and deliver adaptive daily planning. The system combines habit analytics, real-time content recommendations, and automated progress reporting to create a cohesive self-improvement experience. Structured data storage enables precise tracking of user habits, streaks, and completion rates, while intelligent scheduling modules ensure balance across productivity, health, and learning domains.

Core Technologies:

HTML/CSS & JavaScript: Frontend interface for uploading notes, entering queries, and generating quizzes. Provides an interactive and responsive UI for viewing results, asking questions, and managing study materials efficiently.

FastAPI: Lightweight backend framework that handles routing, API requests, and communication between the frontend, tool layer, and memory storage modules. It manages file uploads, query processing, and response generation efficiently.

LLM-inspired Tool Logic: Core intelligent component responsible for selecting appropriate tools based on user queries, retrieving relevant notes, and generating quizzes. It simulates LLM behavior by dynamically deciding actions to provide meaningful and context-based responses.

FAISS (Vector Search):: Retrieves relevant study content from stored notes based on user queries, enhancing learning by providing context-aware and accurate information for effective revision.

In-Memory Data Store (FAISS): Stores embedded study materials and supports fast similarity-based retrieval, ensuring accurate and efficient access to relevant content.

In-Memory Data Store: Stores user routines, habit logs, and completion metrics to support continuous tracking and data-driven insights.

Component-Wise Architecture :

Component	Description
Frontend UI	Built using HTML, CSS, and JavaScript. Allows users to upload notes, ask questions, and generate quizzes. Displays responses in a simple and interactive interface.
FastAPI Backend	Handles routing such as /upload, /ask, and /quiz. Manages file processing, query handling, and communication between components.
Tool Layer	Includes functions like search_notes() and generate_quiz() to perform specific tasks based on user queries.
Embedding Model	Uses Sentence Transformers to convert text into vector embeddings for efficient similarity search.
Memory Store (FAISS)	Stores embeddings of uploaded notes and retrieves relevant content using similarity search.

Query Processing Module	Analyzes user input and selects the appropriate tool (search or quiz generation) using LLM-style logic.
--------------------------------	---

Pre-requisites :

1. Basic knowledge of Python programming
2. Understanding of FastAPI framework
3. Familiarity with NLP and embeddings
4. Installation of required libraries (FAISS, Sentence Transformers)
5. Basic knowledge of HTML, CSS, and JavaScript

Project Workflow :

1. User Access: User opens the AI Study Assistant web interface.
2. File Selection: User selects notes in PDF or text format.
3. File Upload: The file is uploaded through the frontend to the backend.
4. File Processing: Backend reads and processes the uploaded file.
5. Text Extraction: Text content is extracted from PDF or text files.
6. Data Storage: Extracted text is prepared for embedding.
7. Embedding Generation: Text is converted into vector embeddings using Sentence Transformers.
8. Index Storage: Embeddings are stored in FAISS index for fast retrieval.
9. User Query Input: User enters a question or topic.
10. Query Processing: System processes and normalizes the query.
11. Query Embedding: Query is converted into embedding format.
12. Similarity Search: FAISS retrieves most relevant notes based on similarity.
13. Tool Decision: System checks query type (quiz or normal search).
14. Search Execution: If general query → `search_notes()` is called.
15. Quiz Execution: If quiz-related → `generate_quiz()` is called.
16. Content Preparation: Retrieved data is combined and formatted.
17. Response Generation: Final answer or quiz is generated.
18. Output Display: Results are shown on the frontend UI.
19. User Interaction: User can refine queries or generate more quizzes.
20. Continuous Learning: System improves as more notes are uploaded.

1.Front End

Overview:

The frontend of the AI Study Assistant is built using three standard web technologies — HTML for structure, CSS for styling, and JavaScript for interactivity and backend communication. The interface is divided into three main functional sections: uploading notes, asking questions, and generating quizzes. Each section is presented as a card on the page, making the layout clean, minimal, and easy to navigate for students.

The frontend does not rely on any external frameworks or libraries. Everything is handled natively, which keeps the load time low and the codebase straightforward to understand and maintain. The design follows a dark theme that is easy on the eyes during long study sessions.

File: index.html

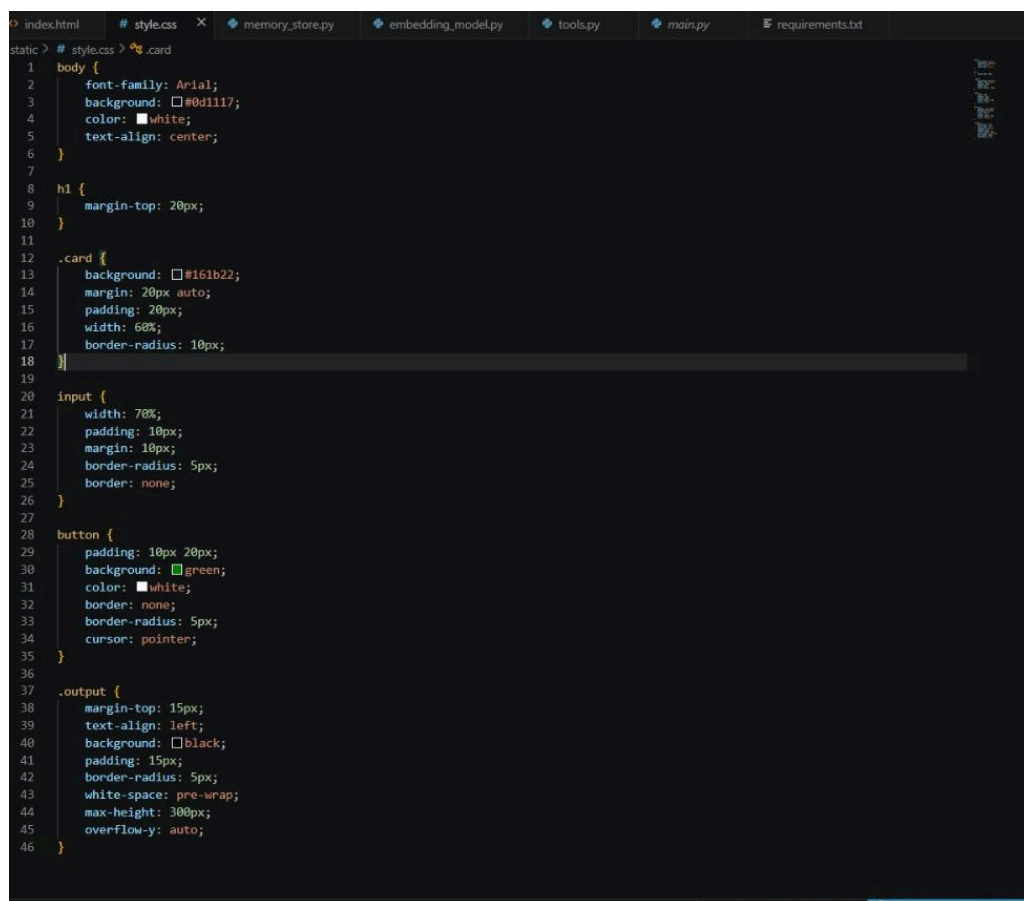
```
2 <html>
3 <body>
4 <div class="card">
5   <h2>Generate Quiz</h2>
6   <input type="text" id="topic" placeholder="Enter topic...">
7   <button onclick="quiz()">Generate</button>
8   <div id="quizBox" class="output"></div>
9 </div>
10
11 <script>
12   async function upload() {
13     let file = document.getElementById("fileInput").files[0];
14     let formData = new FormData();
15     formData.append("file", file);
16
17     await fetch("/upload", {
18       method: "POST",
19       body: formData
20     });
21
22     alert("Uploaded successfully!");
23   }
24
25   async function ask() {
26     let q = document.getElementById("query").value;
27
28     let res = await fetch(`/ask?query=${q}`);
29     let data = await res.json();
30
31     let output = data.results.join("\n\n");
32
33     document.getElementById("answerBox").innerText = output;
34   }
35
36   async function quiz() {
37     let t = document.getElementById("topic").value;
38
39     let res = await fetch(`/quiz?topic=${t}`);
40     let data = await res.json();
41
42     document.getElementById("quizBox").innerText = data.quiz;
43   }
44 </script>
45 </body>
46 </html>
```

```
36 let formData = new FormData();
37 formData.append("file", file);
38
39 await fetch("/upload", {
40   method: "POST",
41   body: formData
42 });
43
44 alert("Uploaded successfully!");
45 }
46
47 async function ask() {
48   let q = document.getElementById("query").value;
49
50   let res = await fetch(`/ask?query=${q}`);
51   let data = await res.json();
52
53   let output = data.results.join("\n\n");
54
55   document.getElementById("answerBox").innerText = output;
56 }
57
58 async function quiz() {
59   let t = document.getElementById("topic").value;
60
61   let res = await fetch(`/quiz?topic=${t}`);
62   let data = await res.json();
63
64   document.getElementById("quizBox").innerText = data.quiz;
65 }
66 </script>
67 </body>
68 </html>
```

This file is the entire frontend of the application. The head section links to the external stylesheet, and the body contains three card sections. The first card handles file uploads — it has a file input and an Upload button that triggers the `upload()` JavaScript function. This function reads the selected file, wraps it in a `FormData` object, and sends it as a POST request to the `/upload` backend route. The second card handles question input, where the user types a query and clicks Ask, which calls the `ask()` function and displays the response in the `answerBox` div below it.

The third card handles quiz generation, where the user enters a topic and clicks Generate, triggering the `quiz()` function. The JavaScript functions `ask()` and `quiz()` both work by reading user input, sending a GET request to their respective backend routes with the input as a URL parameter, parsing the returned JSON response, and updating the appropriate output div on the page. All three functions are asynchronous and handle everything without any page reload, keeping the experience smooth and interactive.

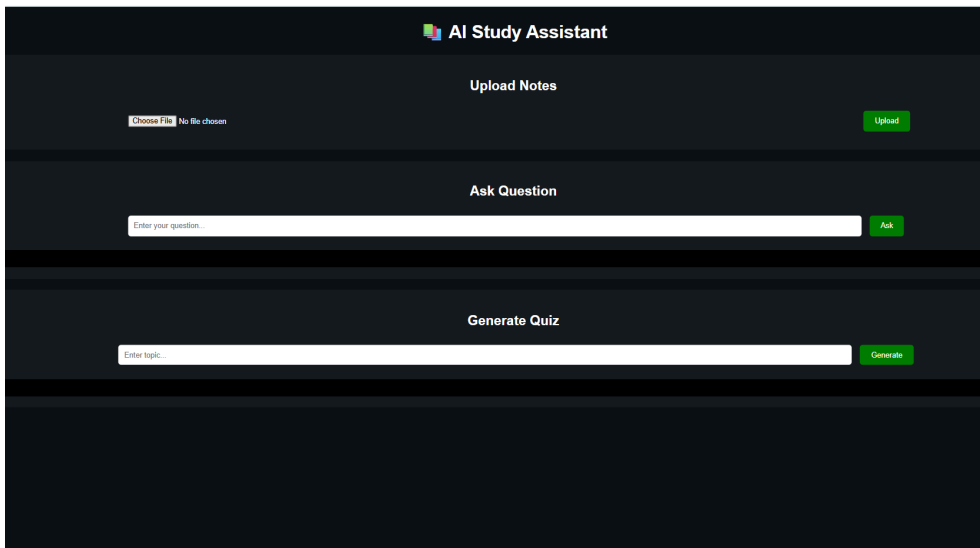
File: style.css



```
static > # style.css > *.card
1 body {
2   font-family: Arial;
3   background: #0d1117;
4   color: white;
5   text-align: center;
6 }
7
8 h1 {
9   margin-top: 20px;
10 }
11
12 .card {
13   background: #161b22;
14   margin: 20px auto;
15   padding: 20px;
16   width: 60%;
17   border-radius: 10px;
18 }
19
20 input {
21   width: 70%;
22   padding: 10px;
23   margin: 10px;
24   border-radius: 5px;
25   border: none;
26 }
27
28 button {
29   padding: 10px 20px;
30   background: #28a745;
31   color: white;
32   border: none;
33   border-radius: 5px;
34   cursor: pointer;
35 }
36
37 .output {
38   margin-top: 15px;
39   text-align: left;
40   background: black;
41   padding: 15px;
42   border-radius: 5px;
43   white-space: pre-wrap;
44   max-height: 300px;
45   overflow-y: auto;
46 }
```

This file handles all visual styling of the application. The body uses a dark background color with white text and centered alignment, giving the page a clean dark-mode look. Each card section has a slightly lighter dark background, rounded corners, centered width, and padding to create a layered panel effect. Input fields are styled to blend into the card background with no visible border, and buttons use a green background with white text to indicate interactivity. The output divs where answers and quizzes appear use a black background, left-aligned text, preserved whitespace for line breaks, and a fixed maximum height with scrolling enabled so long responses remain contained and readable.

Frontend Output:



The screenshot displays the 'AI Study Assistant' web application interface. It features a dark-themed layout with three main functional cards. The first card, titled 'Upload Notes', includes a file selection area with a 'Choose File' button and the text 'No file chosen', alongside a green 'Upload' button. The second card, titled 'Ask Question', contains a text input field with the placeholder 'Enter your question...' and a green 'Ask' button. The third card, titled 'Generate Quiz', has a text input field with the placeholder 'Enter topic...' and a green 'Generate' button. The interface is clean and modern, utilizing a dark background with white text and green accents for interactive elements.

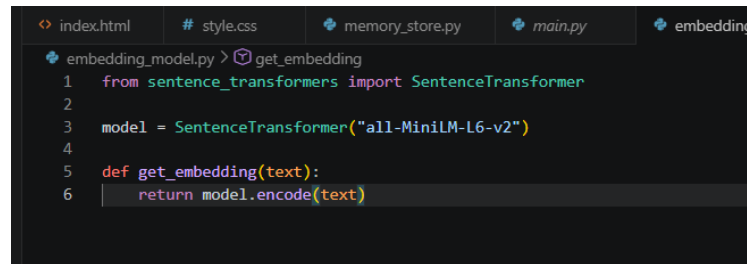
2.Back End

Overview:

The backend is built using FastAPI and is organized across four Python files — `main.py`, `tools.py`, `memory_store.py`, and `embedding_model.py`. Each file has a single clearly defined responsibility. Together they implement a modular pipeline where uploaded text is converted to vector embeddings, stored in a FAISS index, and retrieved semantically when the user asks questions or requests quizzes. The architecture is inspired by real-world LLM tool-orchestration patterns, making it modular, scalable, and easy to extend.

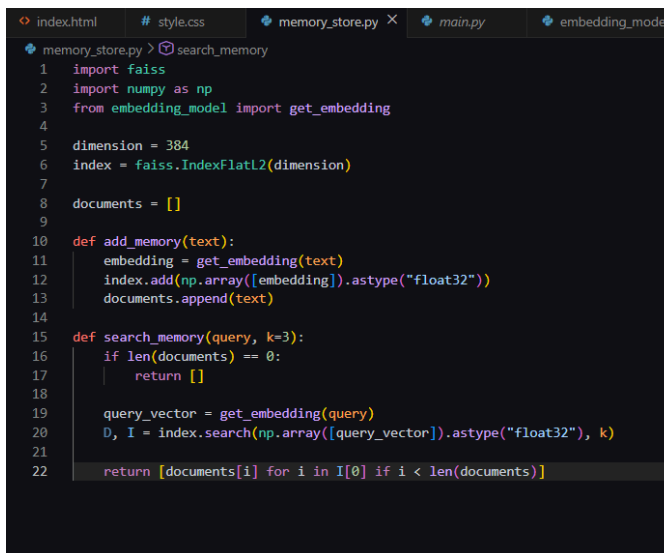
File: `embedding_model.py`

The backend is built using FastAPI and is organized across four Python files — `main.py`, `tools.py`, `memory_store.py`, and `embedding_model.py`. Each file has a single clearly defined responsibility. Together they implement a modular pipeline where uploaded text is converted to vector embeddings, stored in a FAISS index, and retrieved semantically when the user asks questions or requests quizzes. The architecture is inspired by real-world LLM tool-orchestration patterns, making it modular, scalable, and easy to extend.



```
index.html # style.css memory_store.py main.py embedding_model.py
embedding_model.py > get_embedding
1 from sentence_transformers import SentenceTransformer
2
3 model = SentenceTransformer("all-MiniLM-L6-v2")
4
5 def get_embedding(text):
6     return model.encode(text)
```

File: `memory_store.py`



```
index.html # style.css memory_store.py main.py embedding_model.py
memory_store.py > search_memory
1 import faiss
2 import numpy as np
3 from embedding_model import get_embedding
4
5 dimension = 384
6 index = faiss.IndexFlatL2(dimension)
7
8 documents = []
9
10 def add_memory(text):
11     embedding = get_embedding(text)
12     index.add(np.array([embedding]).astype("float32"))
13     documents.append(text)
14
15 def search_memory(query, k=3):
16     if len(documents) == 0:
17         return []
18
19     query_vector = get_embedding(query)
20     D, I = index.search(np.array([query_vector]).astype("float32"), k)
21
22     return [documents[i] for i in I[0] if i < len(documents)]
```

This file manages all storage and retrieval of note content. It initializes a FAISS flat index with 384 dimensions and maintains a parallel Python list called `documents` that holds the original text strings. The `add_memory` function converts incoming text to an embedding and adds it to both the index and the list simultaneously, keeping them synchronized. The `search_memory` function converts a query to an embedding and uses FAISS to find the top three most similar stored documents, returning their original text. This pairing of FAISS for vector search and a Python list for text retrieval is what powers the intelligent question-answering feature.

File: tools.py

This file contains the two core action functions of the system. The `search_notes` function is a clean wrapper around `search_memory` — it takes a query, retrieves relevant results, and returns a fallback message if nothing is found. The `generate_quiz` function takes retrieved note content and returns a formatted set of four questions and answers in a structured Q and A style. In the current implementation the quiz is template-based, but the function is designed so that its content can be replaced with a dynamic LLM call in the future without changing anything else in the system.

```
from memory_store import search_memory

def search_notes(query):
    results = search_memory(query)

    if not results:
        return ["No data available"]

    return results

def generate_quiz(text):
    if not text:
        return "No data available"

    return f"""
Q1: What is the system about?
Answer: It is an online platform connecting customers with restaura

Q2: What problem does it solve?
Answer: It reduces manual errors and improves efficiency.

Q3: Name one feature
```

File: requirements.txt

```
fastapi
uvicorn
sentence-transformers
faiss-cpu
openai
pypdf
jinja2
python-multipart
numpy
```

This file lists all nine Python dependencies needed to run the project. FastAPI is the web framework, uvicorn is the server that runs it, sentence-transformers provides the embedding model, faiss-cpu handles vector search, pypdf reads PDF files, python-multipart enables file upload handling in FastAPI, numpy supports array operations for FAISS, jinja2 provides templating support, and openai is included for potential future LLM integration. Everything can be installed at once with `pip install -r requirements.txt`.

File: main.py

This is the central file that ties everything together. It sets up the FastAPI app, mounts the static directory so the CSS is accessible, and defines the route that serves the HTML page. The upload route accepts a file via multipart form data, checks whether it is a PDF or plain text file, extracts the text content accordingly using either PdfReader for PDFs or direct byte decoding for text files, and then calls `add_memory` to embed and store the content. A success message is returned once the file has been processed and stored.

The ask route receives the user's query, passes it to `search_notes` to retrieve semantically relevant content, and then applies simple keyword-based logic to decide what portion of that content to return. Queries about names return filtered lines, queries about general information return the first 400 characters, and all other queries return the first 500 characters. The quiz route receives a topic, retrieves relevant notes using `search_notes`, passes the content to `generate_quiz`, and returns the formatted quiz as a JSON response. Both routes are concise and rely entirely on the tool and memory layers to do the heavy lifting.

```

index.html  # style.css  memory_store.py  main.py  X  embedding_model.py  tools.py  require
main.py > upload
1  from fastapi import FastAPI, UploadFile, File, Request
2  from fastapi.responses import HTMLResponse
3  from fastapi.staticfiles import StaticFiles
4  from fastapi.templating import Jinja2Templates
5
6  from tools import search_notes, generate_quiz
7  from memory_store import add_memory
8  from pypdf import PdfReader
9
10 app = FastAPI()
11
12 app.mount("/static", StaticFiles(directory="static"), name="static")
13
14 @app.get("/", response_class=HTMLResponse)
15 def home():
16     with open("templates/index.html", "r", encoding="utf-8") as f:
17         return f.read()
18 @app.post("/upload")
19 async def upload(file: UploadFile = File(...)):
20     text = ""
21
22     if file.filename.endswith(".pdf"):
23         pdf = PdfReader(file.file)
24         for page in pdf.pages:
25             text += page.extract_text() or ""
26     else:
27         content = await file.read()
28         text = content.decode("utf-8")
29
30     add_memory(text)
31     return {"message": "Uploaded successfully"}
32
33 @app.get("/ask")
34 def ask(query: str):
35     results = search_notes(query)
36
37     if not results:
38         return {"results": ["No relevant data found"]}
39
40     content = " ".join(results)
41     q = query.lower()
42
43     if "name" in q or "person" in q:
44         lines = content.split("\n")
45         names = [line for line in lines if "Team" in line or "Name" in line]
46         return {"results": names if names else ["No names found"]}
47
48     elif "about" in q or "what" in q:

```

```

47
48     elif "about" in q or "what" in q:
49         return {"results": [content[:400]]}
50
51     return {"results": [content[:500]]}
52
53 @app.get("/quiz")
54 def quiz(topic: str):
55     notes = search_notes(topic)
56     content = " ".join(notes)
57     return {"quiz": generate_quiz(content)}

```

Working of frontend and backend

When a student opens the browser and navigates to the application, FastAPI serves the index.html page. The student selects a PDF or text file and clicks Upload. The JavaScript upload() function sends the file to the upload route. FastAPI reads the file, extracts its text content, calls get_embedding to convert the text into a 384-dimensional vector, stores the vector in the FAISS index, and stores the original text in the documents list. Both are now ready for retrieval.

When the student types a question and clicks Ask, the ask() JavaScript function sends the query to ask. FastAPI converts the query into an embedding using the same model, searches the FAISS index for the three most semantically similar stored documents, applies the keyword-based routing logic, and returns the most appropriate portion of the content as a JSON response. The JavaScript receives this and displays it in the answer box.

When the student enters a topic and clicks Generate, the quiz() function sends the topic to quiz. FastAPI searches the notes, retrieves relevant content, passes it to generate_quiz(), and returns a formatted quiz with four questions and answers. This appears instantly in the quiz output box on the page.

This complete flow — from file upload through embedding generation to vector storage to semantic search to response display — represents a fully functional retrieval-augmented system built from scratch using only open-source tools, without depending on any paid API at its core.

Final Results

AI Study Assistant

Upload Notes

Choose File

Online-Food-Ordering-and-Delivery-System.pdf

Upload

Ask Question

what is this about?

Ask

Online Food Ordering and Delivery System
Team: Madhan, Sheldon, Harsha, Navaneeth
Institution: SRM University AP
Project Overview
Online Platform
Digital food ordering system connecting customers with restaurants
Fast Service
Streamlined process for quick and efficient food delivery
Reliable
Accurate order processing and real-time tracking
Problem Statement
Current Challenges
Manual ordering

Generate Quiz

online food ordering system

Generate

Q1: What is the system about?
Answer: It is an online platform connecting customers with restaurants.
Q2: What problem does it solve?
Answer: It reduces manual errors and improves efficiency.
Q3: Name one feature.
Answer: Real-time order tracking.
Q4: Which backend is used?
Answer: FastAPI.

Conclusion

The AI Study Assistant is a fully functional, AI-powered learning tool that demonstrates how modern NLP techniques can be applied to solve real student challenges. By combining FastAPI for backend routing, Sentence Transformers for semantic understanding, and FAISS for fast vector-based retrieval, the system creates an intelligent pipeline that goes far beyond simple keyword search.

The project successfully achieves its three core objectives — allowing students to upload their study material, retrieve relevant information through natural language questions, and generate quiz content for self-assessment. The modular architecture ensures that each component, from embedding generation to memory storage to tool execution, operates independently and can be improved or replaced without disrupting the rest of the system.

What makes this project particularly meaningful is that it mirrors the design patterns used in real-world LLM-based applications, such as retrieval-augmented generation and tool orchestration, while being built entirely with open-source, locally running components. This makes it cost-effective, privacy-friendly, and accessible to anyone with a standard development environment.

Looking ahead, the system can be extended in several ways — integrating an actual LLM for dynamic quiz generation, adding user authentication, supporting more file formats, or deploying it as a cloud-hosted service. The foundation built here is strong enough to support all of those enhancements. Overall, the AI Study Assistant stands as a practical and well-structured demonstration of how generative AI concepts can be translated into a working, useful application.